

Experiment-17

Name: N.PARNITA

Regno: 192525322

Course Code: MLA0305

Slot:B

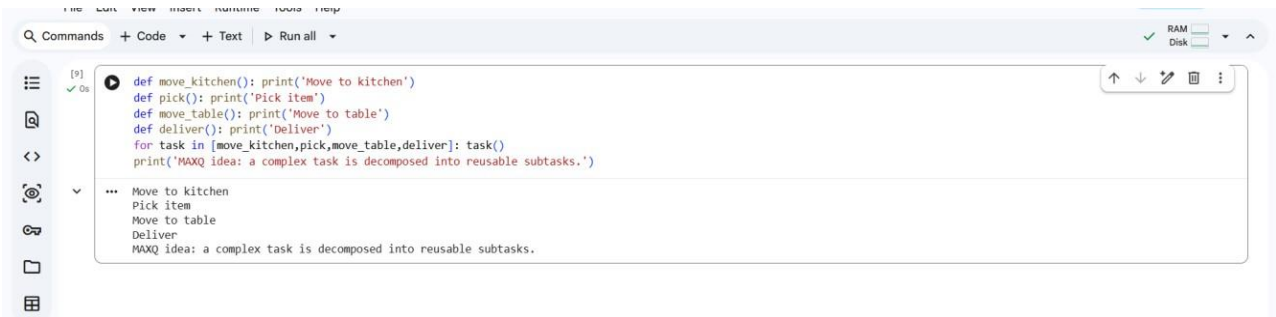
Title: Implementation of Hierarchical Reinforcement Learning Using MAXQ

Aim:

To implement Hierarchical Reinforcement Learning using the MAXQ framework and analyze hierarchical task decomposition.

Procedure:

1. Define a complex sequential decision-making problem.
2. Divide the main task into smaller subtasks.
3. Represent the subtasks using the MAXQ hierarchy.
4. Define primitive actions for lower-level tasks.
5. Define higher-level tasks that combine subtasks.
6. Train the agent at different levels of the hierarchy.
7. Execute subtasks according to the learned hierarchy.
8. Calculate cumulative rewards.
9. Compare hierarchical learning with flat RL.
10. Analyze task decomposition and learning efficiency.



The screenshot shows a code editor with a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The code defines four functions: `move_kitchen()`, `pick()`, `move_table()`, and `deliver()`, each with a corresponding print statement. A loop iterates over these functions, calling `task()` for each. A final print statement outputs the MAXQ idea. The output pane below shows the execution results, including the MAXQ idea.

```
def move_kitchen(): print('Move to kitchen')
def pick(): print('Pick item')
def move_table(): print('Move to table')
def deliver(): print('Deliver')
for task in [move_kitchen,pick,move_table,deliver]: task()
print('MAXQ idea: a complex task is decomposed into reusable subtasks.')
```

```
... Move to kitchen
Pick item
Move to table
Deliver
MAXQ idea: a complex task is decomposed into reusable subtasks.
```

Result:

The MAXQ-based hierarchical agent successfully decomposed the complex task into subtasks. Hierarchical learning improved the organization and efficiency of decision-making.